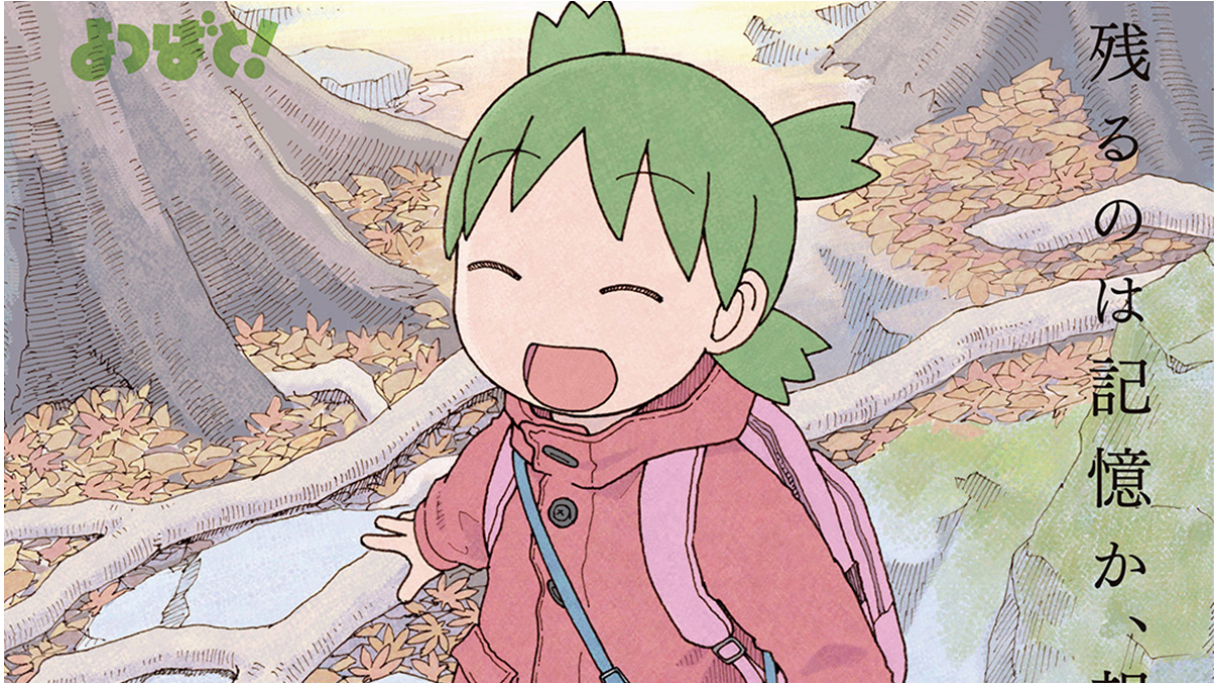


CMSC 216 Notes

Introduction to Computer Systems

Miagao



Yotsuba from Yotsuba&!

Contents

Course Intro	3
Linux Shell Commands	3
C Language I	4
C versus Java	4
C Program Example	4
Compiling a C program	4
Compilation Stages	5
Types in C	5
Numerical Literals	6
Variable Declaration	6
Arithmetic Operators	6
Boolean Type and Operators	7
Precedence and Associativity	7
Assignment Statements	7
Output using printf()	8
Input using scanf()	8
C Control Statements	9

Course Intro

C language is a high-level language close to assembly.

- Exposes how data is organized in a linear address space.
- **Pointers** demonstrate how variables represent locations in memory.
- Shows system calls provided by the OS.

Assembly language shows what hardware can do:

- Instructions processors can execute.
- How function calls work.
- How input/output devices are accessed.

UNIX operating system

- Shows how threads, processes, pipes, etc are building blocks for servers and high-performance applications.
- We will use a variant of UNIX called **Linux**.

Linux Shell Commands

Flags are extra options for commands. They are generally denoted by a dash.

The following commands are the *essentials* for writing and executing C programs.

- 1) `pwd` displays the current direction. Forward slashes separate directories and files.
- 2) `ls` displays all files and directories in the current directory.
 - a) `ls -al` will also include hidden directories and files.
 - b) `ls -F` will append specific characters to items' names depending. Particularly, directories are marked with a forward slash.
- 3) `cd` changes the directory to the home directory.
 - a) `cd DIRECTORY_NAME` changes the directory to the specified directory.
 - b) `cd ~` makes the current directory the home directory.
- 4) `mkdir` creates a new directory in the current directory.
- 5) `cp FILE_1 FILE_2` will create a new file using the data of the first file.
- 6) `rm FILE` permanently removes the specified file.
- 7) `mv` moves files/directories to a directory or renames files/directories.
 - a) `mv FILE_1 FILE_2` renames the first file to `FILE_2`.
 - b) `mv FILE DIRECTORY` moves the file into the specified directory.
 - c) `mv DIRECTORY_1 DIRECTORY_2` renames the first directory to `DIRECTORY_2` if the second directory doesn't exist. Otherwise, the first directory is moved to the second directory.
- 8) `rm DIRECTORY` removes the specified *empty* directory.
 - a) `rm -f -r DIRECTORY` removes the specified directory, *even if it is non-empty*.

C Language I

For this course, we will be using C99.

- C is not object oriented, so instead of the term *method*, we use **function**.
- To stop a C program, use ctrl + c.

C versus Java

- C is more performant because it trusts the programmer to manage details such as array bounds, memory leaks, and error checking.
 - Java's garbage collection and constraints hurt performance.
- C is the next level after assembly, thus it's used for writing operating systems such as Unix.
- C compiles to machine code which the processor can directly execute. Java requires an intermediate step of converting to bytecode.
- As mentioned prior, C lacks explicit object-oriented features.

C Program Example

```
1  #include <stdio.h>
2  int main() {
3      printf("Fear the turtle\n");
4      return 0;
5  }
```

- `#include` allows the compiler to include additional header files in the program. In this case, the statement is needed to use `printf()`.
- The `main()` function is where all C programs begin execution.
 - Return values from this function indicate the end of a program.
 - By convention, an exit code of 0 means that everything went well.

Compiling a C program

C programs must be compiled to be executed.

- The **gcc** program is will be used to build our programs.
 - `gcc SOURCE_FILES`
- The common flags for gcc are as follows:
 - `-g` enables debugging.
 - `-Wall` warns about potential problems.
 - `-o FILENAME` places the executable in the specified file.
 - `-c` compiles to object file (without link).

The following are examples of compiling single files and multiple files at once.

```
1 gcc file.c
2 gcc -g -Wall -o prog1 prog1.c tools.c
```

UNIX

To run executables in UNIX, type the name of the executable. In some cases, the executable's name should be preceded by `./`.

Compilation Stages

- **Preprocessor**

- *Does text-processing of source code.*
- Used to provide declarations shared by different source code.
- Directives begin with the pound sign, #, but do not end with a semicolon.

- **Translation**

- *An object file (.o) is created.*
- Ensures individual files are consistent within themselves.

- **Linking**

- *Joins one or more compiled object files together.*
- *Creates an executable file.* Note that by UNIX convention, executable files lack a filename extension

Types in C

- char, short, int, long int.
 - These can be signed or unsigned.
- float, double
- * is a pointer to a variable.
- [] represents arrays of variables.
- union {} represents variables sharing the same space.
- struct {} represents the concatenation of variables.
- enum {} represents named integer values.

Type name	Minimum size (bytes)	Size on <i>grace</i> (bytes)
char	1	1
short	2	2
int	2	4
long	4	8
float	4	4
double	8	8
long double	10	16

Numerical Literals

Suffixes can be used to specify a numerical literal's type.

- "L" can be used to specify "long."
- "f" can be used to specify "float."
 - By default, a decimal is a double.

Prefixes can be used to specify bases other than base 10.

- "0" represents octal.
- "0x" represents hexadecimal.

Variable Declaration

Variables may be declared as follows.

```
typename var1, var2, ...;
```

Variables may also be initialized during declaration.

```
1  int x = 6;
```



Even if an initial value is not given, declaration always reserves space for the variable in memory. In the example above, 4 bytes will be reserved even if the initial value of 6 was not given.

- However, it is recommended to give an initial value, as C will not stop you from using an uninitialized variable. In fact, it will use whatever data is in the reserved memory space.

Arithmetic Operators

The basic arithmetic operators in C are analogous to Java.

- + add
- - subtract
- / divide
- % remainder
- * multiplication

Compound assignment operators are basic arithmetic operators followed by an equals sign.

- +=, -=, /=, %=, *=
- ++ increment
- -- decrement
- **(typename)** casting

 Note

Integer division occurs only if both operands are integers. Otherwise, floating point division occurs.

Boolean Type and Operators

There is no boolean type in C. Integers are used instead.

- 0 represents **false**.
- Any other integer represents **true**. By convention, this is 1.

Similar to arithmetic operators, the boolean operators are analogous to Java.

- Relational operators: <, >, <=, >=
- Equality operators: ==, !=
- Logical operators: &&, ||, !

Because integers are used instead of a true boolean type, these either evaluate to **0 (false)** or **1 (true)**

```
1  int i;  
2  i = (!3) == (4 < 2);  
3  i = (!2) || (5 && i);
```

```
1  boolean i;  
2  i = (!true) == (4 < 2);  
3  i = (!true) || (5 && i);
```

Precedence and Associativity

some of these fellas go first. Lowk PEMDAS.

Assignment Statements

The **assignment operator** in C is the equals symbol, =.

- It is right associative. A value is returned from the operator to the variable on the left side of the assignment operator.
- *Note, this means that variable assignment is an expression. Consequently, you can write any expressions as individual statements.*

Generally, expression statements are most useful with the assignment operators.

```
1  int x, y;  
2  y = x = 123; //x is assigned to 123, then y is assigned to 123  
   transitivity through x.  
3  y = 5 + (x = 3); //x is assigned to 3, then y is assigned to 5 + x,  
   which is 8.
```

Output using printf()

The `printf()` function is formally declared in `stdio.h`. It can print strings, which may be formatted with a variable number of expression statements.

```
printf("formatstring", expression1, expression2, expression3, ...);
```

The expressions are inserted into the format string using format specifiers. These correspond to each passed-in expression, and decide how each value will be interpreted.

- `%d` prints an integer in decimal.
- `%ld` prints a long integer.
- `%u` prints an unsigned integer.
- `%f` prints a floating point value.
- `%c` prints a character.
- `%s` prints a character string.
- `%x` prints in hexadecimal.
 - `%X` makes the letters capital.
- `%e` prints in exponential form (e notation).

Like Java, **escape sequences** can be used in strings to specify things that would be otherwise inconvenient to type out.

- `\n` creates a new line.
- `\t` creates a horizontal tab.

Input using scanf()

The `scanf()` function is formally declared in `stdio.h`.

```
scanf("formatstring", &variable1, &variable2, &variable3, ...);
```

The format string specifies the inputs being read using format specifiers and inserts them into each passed-in variable.

- Format specifiers should match the type of their corresponding variables. If this is not the case, the function will cease.
 - *Most* specifiers skip leading whitespace.
- Whitespace in the format string simply skips any whitespace input at that point.
- Other characters just tell the function to read the corresponding characters in the input stream.

The `scanf()` function **returns the number of items assigned to variables or the EOF (End of File Marker)**.

Note

The ampersand symbols associated with each variable means that the address of each variable is being retrieved.

C Control Statements

Just like Java, C has **if/else/else if**, **for**, **while**, and **switch** statements.

The flow of loops can be controlled using the following keywords:

- **break** immediately ends a loop.
- **continue** skips the remainder of the loop and advances to the next iteration.

Compound statements are delimited by {}, and can go anywhere a single statement can. Variables defined within are limited to the scope of the compound statement.